M2104 Object-Oriented Design & Modelling – Course Text – Preparing a Program
Session 3

## *1 Lesson: actors, events, use cases*

### *1.1 Goal: Design*

The goal of this part of the course is to start the proper *design* process, that is, to prepare the specifications of a software to be developed (instead of transcribing parts of software programs that already exist, as in the first two sessions). The design process can start with a *case study*: the analysis of items that transpire from the expression given by a short text (in a professional setting, by the project's *requirements*). The approach given here is to identify the *actors* interacting with the software, the *events* that will allow the actors to communicate with the software, and the *use cases* of the software (what are the features offered to the actors).

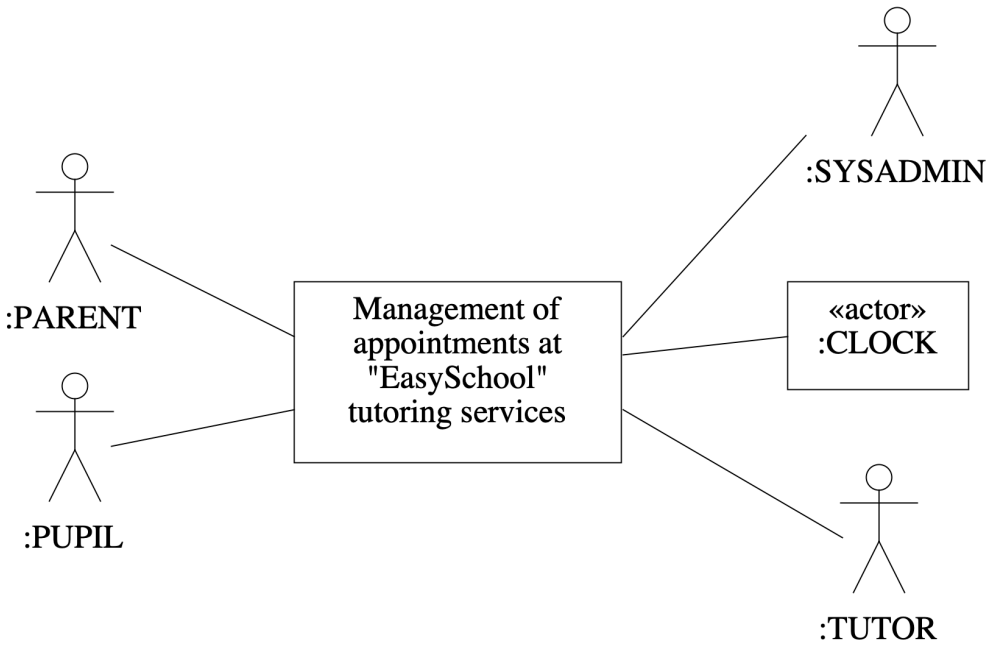
### *1.2 External actors*

The first order of business is to specify the *primary actors*: they are the reason that the program exists, its *end-users*. The customers on a commercial website, the player of a video-game, the user of a smartphone app, the members of a social network… at therefore the person or people for whom the software is developed. They are most of the time *human* actors. We can then specify the *secondary actors*, that will aid the software to perform its intended functions: the administrators of a website, the moderators of a network, &c. There can also be *non-human actors* such as external web-services (for geolocation, payment…), a clock (representing access to the passage of time in the environment), databases, &c. All of these are the *external actors*, representing all sources and destinations of exchanges with the software to design; their list is more important that the fact of being primary or secondary (which remains sometimes subject to interpretation).

In the UML diagrams (detailed further on), actors are represented by a human figure (*stickperson*) for the humans and by a rectangle box containing the *actor* stereotype for non-humans, with their names in UPPERCASE preceded by : (column). The name of an actor represents the *type of actor*; in the description of a specific case, an individual name can also be used as in `Alice:ADMIN` or `Bob:USER`.

**Warning!** External actors are by definition external to the software, they are not objects (and, in the case of real-life software design, it will be actual people…). This does not preclude the existence of classes representing the data of a *user account*, for example, but these are not on the same ontological plane.

After having listed external actors, we can produce a *context diagram*; this is not a UML diagram proper, but a tool useful to represent the general context of a project. Actors identified as *primary actors* are placed on the left, secondary actors on the right, and a rectangle box representing the software being designed (in its entirety) in the middle, as in here:




All actors are linked to the software by simple line (with no other decoration). The interaction between actors outside of the software are never part of the design of the software.

### *1.3 Events*

In this course, *events* are the information flows between external actors and the software (application, website, information system…) being designed. We distinguish between:

- *external events*, from an external actor to the software, and
- *result events*, from the software to an external event.

An *external event* is an action performed by an external actor on the software, whether using its graphical user interface (mouse-click on a control) or another input (keyboard, controller, voice input). The software has to take this event into account by performing one of its *features*.

A *result event* is information sent by the software to an external actor, whether on the display (text or graphic message), or by the means of an effect on the environment (sound, video, file creation, electronic mail sent, external library call…). This can be the result of a sequence of actions started by an external event.

Listing all of the informations that are envisioned as possible exchanges between the external actors and the software (i. e., the events) is the first step in characterising *what the software can do* from an external viewpoint (without trying to envision how it will function internally yet).

### *1.4 Use cases*

A *use case* for some software represents the *usage of a feature of the software by an external actor*. The list of use cases is this the list of *all actions that can be performed by an external actor using the software*. A use case is generally a verb phrase (written with an action verb), given from the point of view of the actor, such as *Add an item to the basket*, *Press the Validation button*, *Place a piece on the game board*, &c. Use cases can be described by *user stories*, describing what is expected in each case in plain text.

Different *scenarii* can be envisioned for a single use case: the *nominal scenario* is what happens in standard conditions, there can also be one or several *alternate scenarii* for specific conditions, and one or several *error* or *exception scenarii* if a problem occurs. For example, when considering a website using a registration form, the use case *Register a new user* can have as its nominal scenario the addition of the information entered in a new user account, as an alternate scenario an absence of action if the user was already registered, and as an error scenario the display of a message when the submitted information is wrong (wrong format for address/phone number, missing fields, &c.).

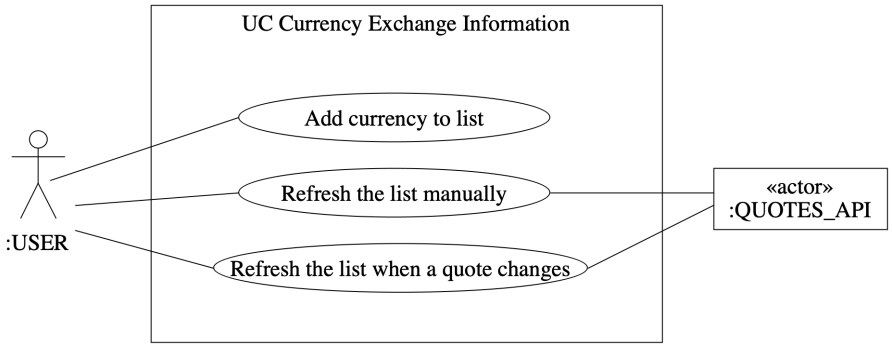
For each use case, **one and only one single external event** from an external actor triggers that use case. It can result (or not) in one or several **result events**.

### *1.5 The Use Case Diagram*

The **use case diagram** is a UML diagram presenting the use cases of some software and the actors involved. There are several versions, we will only use the bases in this course. The diagram includes a **large rectangle box with a title** (such as *UC for software X*) with all of the use cases represented by **ovals** containing their description, and outside, the **external actors involved** with each use case. Each actor responsible for **triggering the external event** starting the use case, as well as the actors receiving a **result event** following any scenario for this use case (if there are result events), are linked to the use case (by a simple line).

In order to simplify the diagram (by reducing the number of crossed lines), actors may be duplicated in this diagram. Moreover, it is possible to hierarchise the use cases by producing a **use case package diagram**, and one use case diagram per package.

In the following example, a simple app allows a user to track exchange rates between the currencies they chose; the app calls on an external, non-public service in order to get the relevant data. Three use cases: the user may add a currency to their list, request that the list is refreshed, and the external service can send an updated quote when some exchange rate has changed. In the first case, there is no result event. In the second, a request is sent to the external service, which may or not trigger the last use case. In that last use case, the page is refreshed for the user. This is the corresponding diagram:



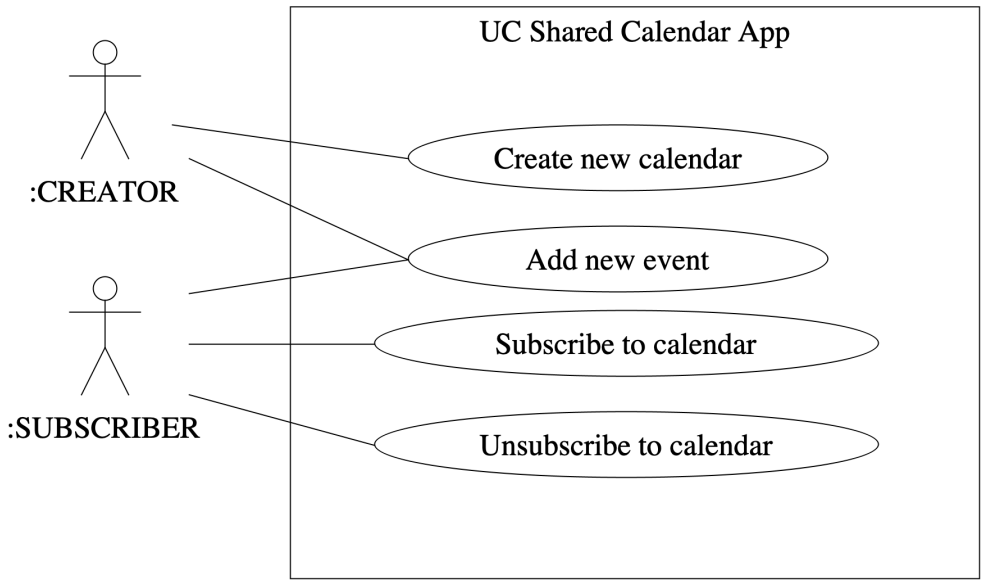
## 2 *Example – Case Study*

This is a first example of a *case study*:

| | |
|---|---|
| **Theme** | The objective is to develop a **shared calendar** app. A user can create a new calendar (agenda) with a name, receiving a link that they can send to other users; the latter may then subscribe to the calendar, and unsubscribe when wanted. The creator of the calendar may add events to it (date and description). Each subscriber receives the new events. |
| **Not required** | Modification/deletion of events, time handling (reminders, alarms), duration/repeat events, adding events in a calendar not belonging to the user. |
| **Required** | Characterise the software, list the external actors, the external and result events, produce a use case diagram. |

Upon reading the text, it appears that the software to be designed involves a **multi-user application**. The people using the calendar can have two kinds of **role**, the one of calendar creator or of subscriber to a calendar; these roles corresponds to the **types of external actors** that can be named `:CREATOR` and `:SUBSCRIBER`. Note that it is possible that a person can change roles (for the same application, a calendar creator can also subscribe to other calendars). External events are also clearly defined. A creator may: crate a new calendar, add new events; a subscriber may: subscribe or unsubscribe to a calendar. We can then deduce the result events: a calendar creator receives the link to send for subscribing to their calendar, when creating a new calendar; a subscriber receives a notification when an event has been added to one of the calendars they are subscribed to.

This is the use case diagram:




## 3 *Exercise – Case Study*

Now perform the same work on the following theme:

| | |
|---|---|
| **Theme** | The objective is to develop a **micro-blogging system**. Users create a profile (with a unique member name) on their first connection. Afterwards, members may follow other members or stop following them. Each member may then post a short (text) message, which will be sent to all that follow them at that time. |
| **Not required** | Message deletion and edition, reactions and responses. |
| **Required** | Characterise the software, list the external actors, the external and result events, produce a use case diagram. |